UNIVERSIDADE MOGI DAS CRUZES
ENGENHARIA DE SOFTWARE

ANDRÉ QUINTINO AIRES AFONSO
CARLOS VINICIUS BORBA RIBEIRO
KAUET YUDI SUGAWARA BRITO

ESTEIRA DEVSECOPS

OWASP ZAP, Bearer e Trivy aplicados ao projeto EmprestaCampus

Mogi das Cruzes
2026

ANDRÉ QUINTINO AIRES AFONSO
CARLOS VINICIUS BORBA RIBEIRO
KAUET YUDI SUGAWARA BRITO

ESTEIRA DEVSECOPS

OWASP ZAP, Bearer e Trivy aplicados ao projeto EmprestaCampus

Relatório técnico do projeto EmprestaCampus apresentado ao curso de Engenharia de Software da Universidade Mogi das Cruzes, como requisito parcial de avaliação.

Mogi das Cruzes
2026

SUMÁRIO

1 INTRODUÇÃO E CONTEXTUALIZAÇÃO
2 DESCRIÇÃO DAS FERRAMENTAS ADOTADAS
2.1 SAST — Semgrep OSS
2.2 SCA — Trivy
2.3 Secrets scanning — TruffleHog
2.4 Ferramentas complementares da esteira avançada
3 INTEGRAÇÃO COM OS REQUISITOS DO PROJETO
6 CONCLUSÃO E RECOMENDAÇÕES
REFERÊNCIAS TÉCNICAS E REPOSITÓRIO
APÊNDICE A — EVIDÊNCIAS VISUAIS

1 INTRODUÇÃO E CONTEXTUALIZAÇÃO

A segurança incorporada ao ciclo de desenvolvimento (SecDevOps/DevSecOps) desloca verificações de segurança para o fluxo normal de integração e entrega de software. Ao automatizar análises em cada push e Pull Request, a equipe consegue detectar problemas antes da integração à branch principal, reduzir dependência de inspeções manuais e manter rastreabilidade das validações.

No EmprestaCampus, aplicação web de gerenciamento de empréstimos de equipamentos acadêmicos, a automação é relevante porque o sistema trata autenticação, dados pessoais, controle de acesso, empréstimos, notificações e registros de auditoria. Uma pipeline integrada ajuda a detectar exposições de segredos, vulnerabilidades em componentes, padrões inseguros no código e falhas observáveis em uma aplicação em execução.

O modelo adotado combina verificações preventivas (SAST, SCA e secrets scanning) com verificações dinâmicas (DAST), privacidade e inventário de componentes (SBOM). O fluxo é acionado para pushes e Pull Requests direcionados à main, conforme documentação e workflow presentes no repositório.

2 DESCRIÇÃO DAS FERRAMENTAS ADOTADAS

2.1 SAST — Semgrep OSS

Finalidade: análise estática do código-fonte, identificando padrões de código associados a vulnerabilidades sem necessidade de executar a aplicação. O workflow gera relatório SARIF, executa etapa bloqueante para achados de severidade ERROR e publica os resultados no GitHub Code Scanning.

Justificativa: Semgrep oferece regras comunitárias/padrões de segurança configuráveis, feedback rápido e integração com Actions e Code Scanning.

2.2 SCA — Trivy

Finalidade: análise de vulnerabilidades de componentes e pacotes. Na pipeline principal, o projeto constrói a imagem Docker do backend e analisa vulnerabilidades HIGH/CRITICAL com correção disponível, conforme a descrição no README. O relatório completo em JSON é publicado como artefato.

Justificativa: Trivy é um scanner open-source que pode analisar imagem de container e dependências de sistema/aplicação; a execução sobre a imagem construída oferece um contexto próximo do artefato que seria distribuído.

2.3 SECRETS SCANNING — TRUFFLEHOG

Finalidade: procurar tokens, senhas, chaves e outros segredos expostos no histórico Git e em commits novos. O workflow utiliza checkout do histórico completo e separa verificações do histórico e do conjunto de commits novos; relatórios são disponibilizados como artefatos.

Justificativa: a inspeção de histórico completo é importante pois uma credencial pode permanecer em commits anteriores mesmo depois de removida do arquivo atual. TruffleHog detecta credenciais por verificadores e pode distinguir achados verificados e não verificados.

2.4 FERRAMENTAS COMPLEMENTARES DA ESTEIRA AVANÇADA

OWASP ZAP (DAST): testa a aplicação em execução, buscando falhas de segurança web, tais como problemas em cabeçalhos, exposição e padrões exploráveis. O workflow cria ambiente efêmero com PostgreSQL, Django e Nginx, prepara TLS e aguarda a aplicação responder por HTTPS antes do Full Scan. O arquivo .zap/rules.tsv controla quais alertas são bloqueantes.

Bearer: análise de segurança e privacidade orientada a identificação de dados pessoais no código, apoiando a revisão de exposição de PII e aspectos relacionados à LGPD. A esteira publica relatórios HTML e inventário de PII.

Trivy SBOM: produz inventário de componentes no formato CycloneDX, útil para rastrear dependências e apoiar resposta a vulnerabilidades e gestão de supply chain. O job é descrito no README como informativo/não bloqueante.

Categoria	Ferramenta	Escopo	Critério bloqueante documentado
SAST	Semgrep OSS	Código-fonte e padrões de vulnerabilidade	Achados de severidade ERROR
SCA	Trivy	Imagem Docker backend, pacotes e CVEs	HIGH/CRITICAL com correção disponível
Secrets	TruffleHog	Histórico Git e commits novos	Segredo verificado no histórico ou qualquer achado nos commits novos, conforme README
DAST	OWASP ZAP	Aplicação Django/Nginx executada em HTTPS	Alertas marcados FAIL em .zap/rules.tsv
Privacidade	Bearer CLI	Riscos e inventário de PII no código	Achados critical/high/medium, segundo README
SBOM	Trivy CycloneDX	Inventário de componentes	Job informativo; publica artefato

3 INTEGRAÇÃO COM OS REQUISITOS DO PROJETO

A integração da pipeline com os requisitos do sistema ocorre por meio de verificações repetíveis e registradas. As análises não substituem testes funcionais nem auditoria de requisito individual, mas fornecem uma camada de controle contínuo que ajuda a manter os mecanismos de segurança implementados.

Área/requisito documentado no projeto	Como a pipeline contribui	Limite da verificação
Autenticação, login, 2FA e recuperação de senha	Semgrep analisa padrões de código; DAST testa superfície web; TruffleHog busca segredos expostos	Não prova por si só todos os cenários de autenticação; testes funcionais dedicados continuam necessários
Proteção de senhas e dados pessoais (PBKDF2/Fernet)	SAST e DAST podem identificar padrões inseguros; Bearer auxilia a localizar tratamento de PII	Scanner não comprova criptografia correta nem gerenciamento operacional da chave
Uso de SECRET_KEY/ENCRYPTION_KEY via ambiente e não versionadas	TruffleHog detecta possíveis chaves/credenciais expostas no Git e histórico	Rotação/revogação de segredo comprometido deve ser realizada manualmente e documentada
HTTPS/TLS, cookies, CSRF, HSTS e cabeçalhos	ZAP observa configuração e comportamento da aplicação em execução	Resultado depende do ambiente, autenticação, regras ZAP e cobertura de rotas
Controle de acesso e papéis de usuário	SAST/DAST podem apontar padrões ou rotas suscetíveis; PR checks reforçam revisão antes da integração	Não substitui matriz de autorização nem testes de acesso por papel
Auditoria e registro de operações	Pipeline cria trilha de execução, status dos checks e relatórios; protege integração na main	É a trilha do processo CI; não equivale aos logs de auditoria de negócio no PostgreSQL
Dependências e imagem Docker	Trivy verifica componentes e vulnerabilidades; SBOM CycloneDX inventaria os componentes	Mitigação de CVE requer atualização, rebuild e nova análise
Integridade do fluxo de desenvolvimento	Ruleset exige PR e três checks de segurança obrigatórios; impede exclusão e force push	A regra de aprovação de reviewer está configurada com 0 aprovações requeridas

Dessa forma, a automação auxilia na conformidade ao tornar as verificações de segurança mais frequentes, auditáveis e associadas à alteração de código. Ela reduz a chance de que uma alteração com falha detectada pelos scanners seja incorporada à main sem passar pelos controles estabelecidos.

6 CONCLUSÃO E RECOMENDAÇÕES

A análise do repositório e das evidências aponta que a implementação da pipeline DevSecOps está operacional: os jobs principais e complementares consultados tiveram conclusão success, a branch main possui Ruleset ativo com checks requeridos e há uma evidência de PR bloqueada pela falha de TruffleHog. A arquitetura de controles combina detecção automatizada, relatórios como artefatos, publicação de resultados no Code Scanning e gate de integração.

Conclusão: o EmprestaCampus possui uma esteira de segurança integrada ao GitHub Actions com mecanismos de prevenção e detecção contínua, e a proteção da branch main impede a integração de alterações quando os checks de segurança obrigatórios não são aprovados.

REFERÊNCIAS TÉCNICAS E REPOSITÓRIO

- Repositório EmprestaCampus: https://github.com/carlosvbr/EmprestaCampus
- Workflow DevSecOps Pipeline: .github/workflows/devsecops-pipeline.yml
- Workflow DevSecOps Avançado: .github/workflows/devsecops-avancado.yml

• README do projeto (descrição dos jobs, critérios e execução local)

• Ruleset Devops2: https://github.com/carlosvbr/EmprestaCampus/rules/26828817

• GitHub Actions run 42560100941 (Pipeline principal)

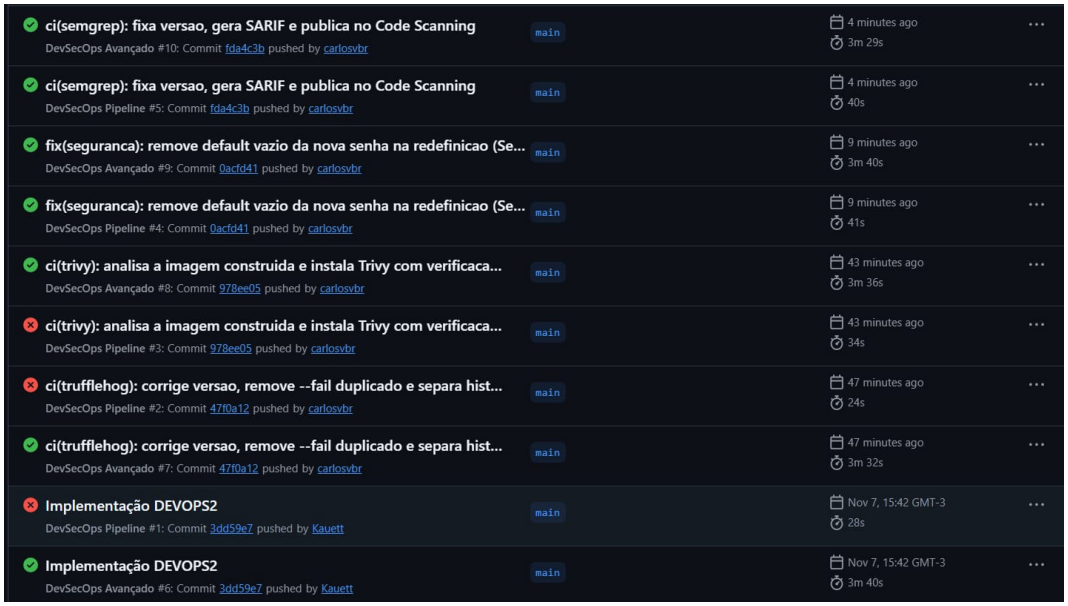
• GitHub Actions run 42560101039 (Pipeline avançada)

• Pull Request #43: https://github.com/carlosvbr/EmprestaCampus/pull/43

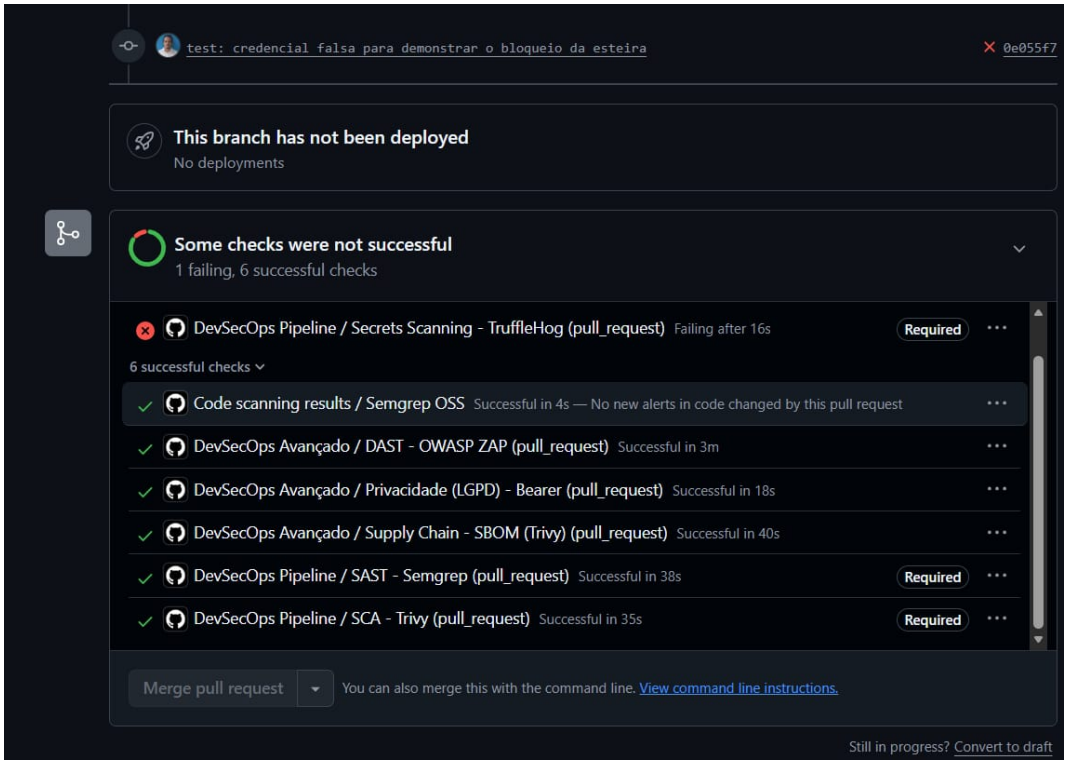
• Capturas de tela fornecidas pelo usuário no arquivo Evidencias.zip (13 arquivos JPG)

APÊNDICE A — EVIDÊNCIAS VISUAIS FORNECIDAS

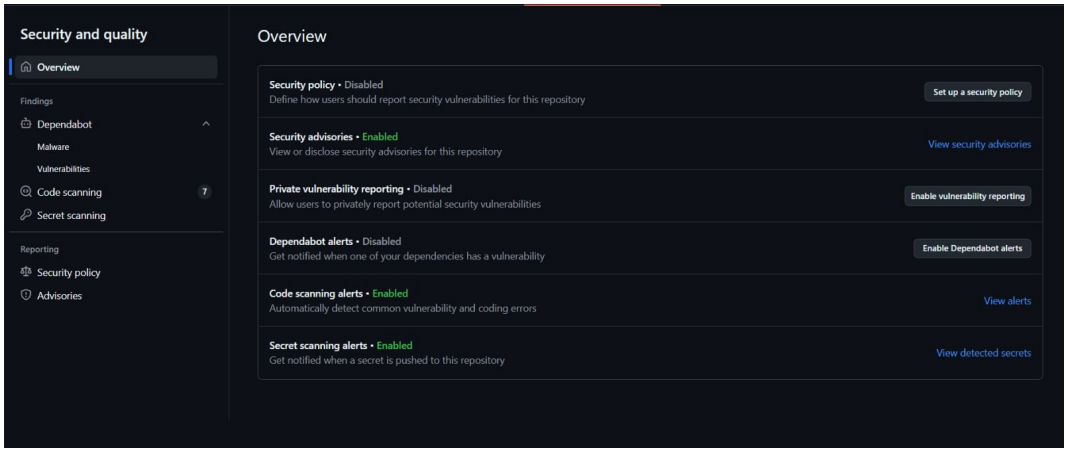
Evidência 01 — Histórico de execuções de workflows DevSecOps no GitHub Actions.



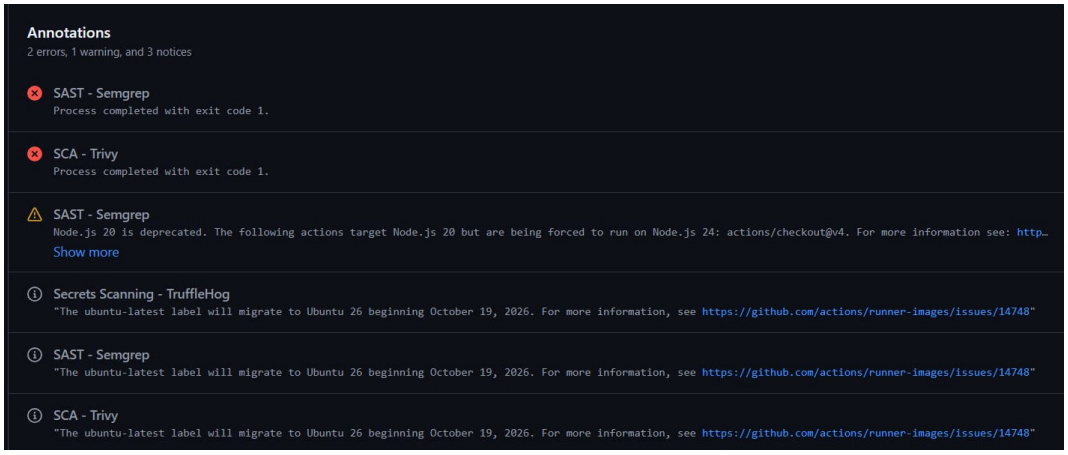
Evidência 02 — Resumo de Pull Request com checks de validação concluídos.



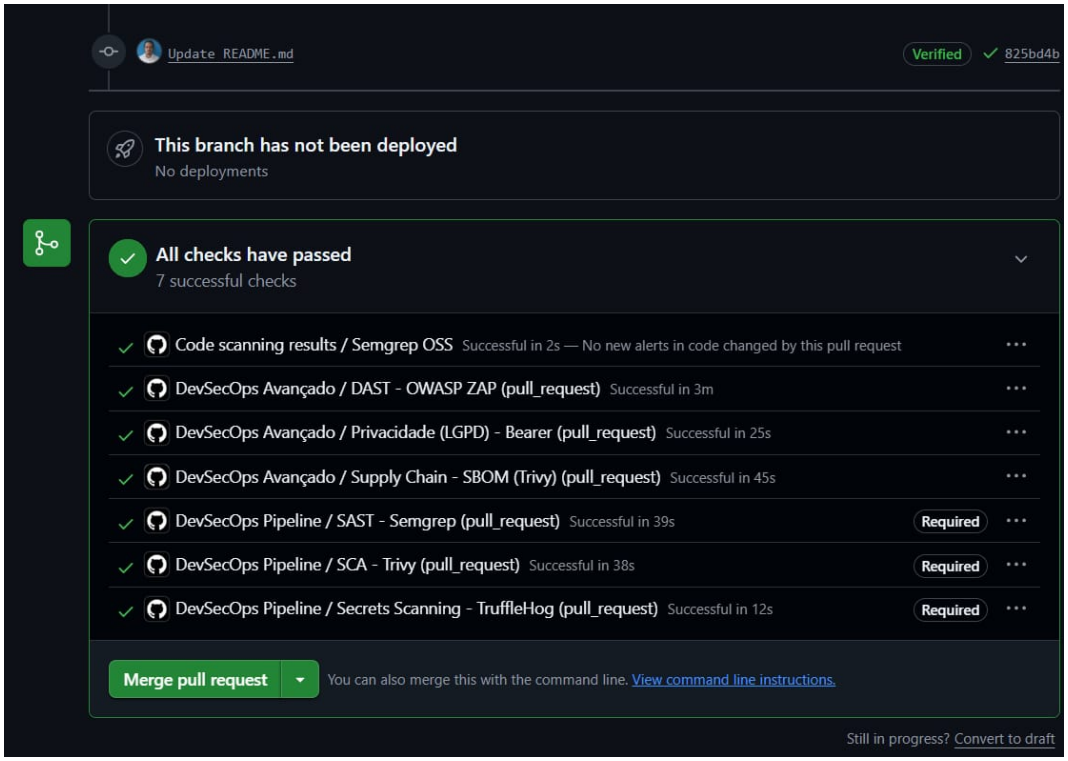
Evidência 03 — Visão geral de Security and quality no repositório.



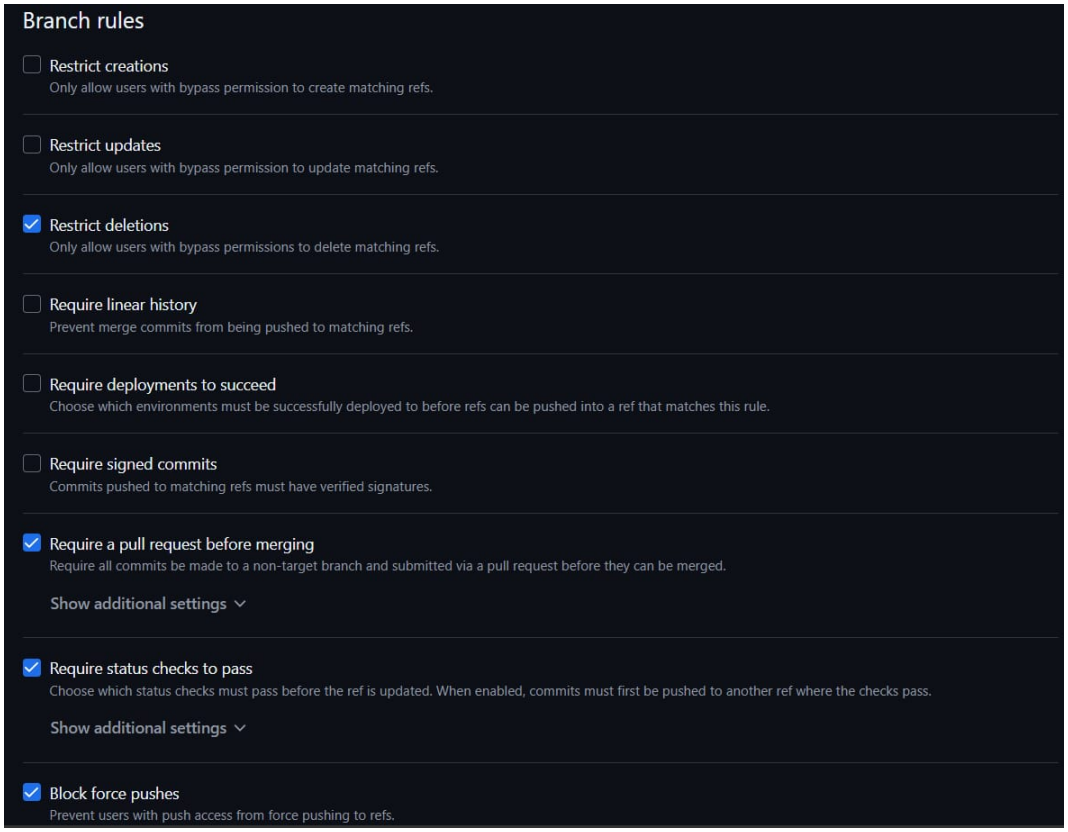
Evidência 04 — Registro de jobs e etapas dos workflows de segurança.



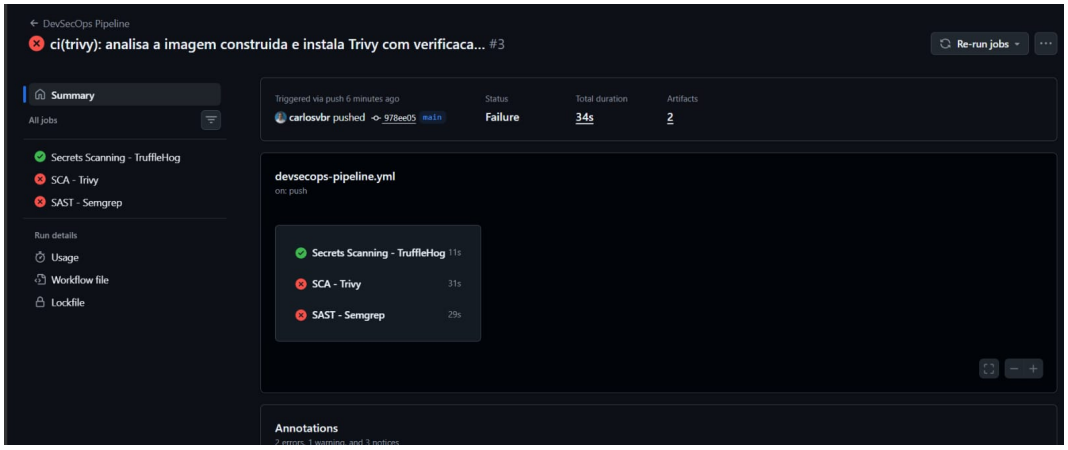
Evidência 05 — Pull Request com sete checks aprovados; os três checks da pipeline principal estão marcados como Required.



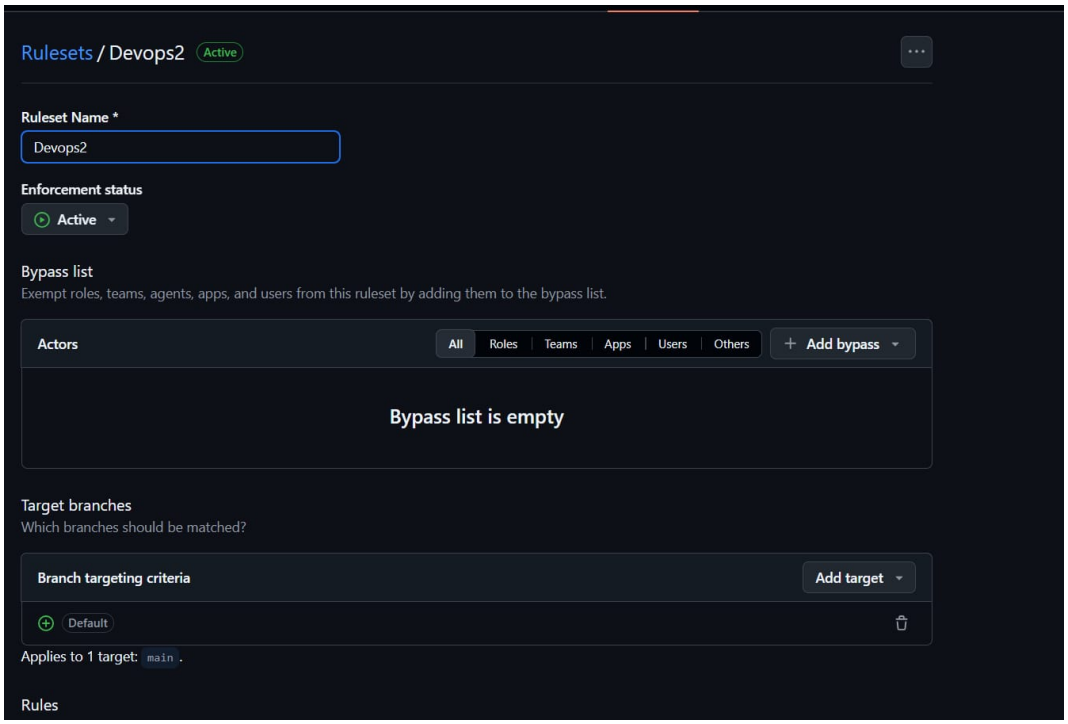
Evidência 06 — Tela de configuração de segurança e regras do repositório.



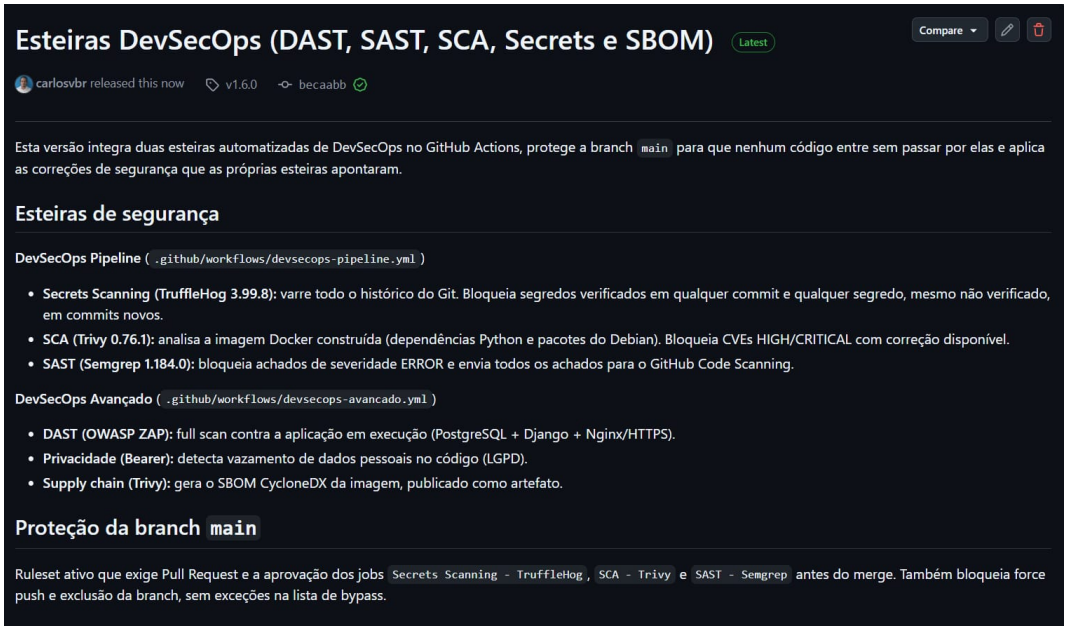
Evidência 07 — Execução de workflow com conclusão positiva e lista de jobs.



Evidência 08 — Tela de criação/configuração do Ruleset de proteção de branch.



Evidência 09 — Trecho de documentação do projeto sobre esteiras DevSecOps e proteção da main.



Esteiras DevSecOps (DAST, SAST, SCA, Secrets e SBOM) Latest

Compare

carlosvbr released this now v1.6.0 becaabb

Esta versão integra duas esteiras automatizadas de DevSecOps no GitHub Actions, protege a branch `main` para que nenhum código entre sem passar por elas e aplica as correções de segurança que as próprias esteiras apontaram.

Esteiras de segurança

DevSecOps Pipeline (`.github/workflows/devsecops-pipeline.yml`)

- **Secrets Scanning (TruffleHog 3.99.8):** varre todo o histórico do Git. Bloqueia segredos verificados em qualquer commit e qualquer segredo, mesmo não verificado, em commits novos.
- **SCA (Trivy 0.76.1):** analisa a imagem Docker construída (dependências Python e pacotes do Debian). Bloqueia CVEs HIGH/CRITICAL com correção disponível.
- **SAST (Semgrep 1.184.0):** bloqueia achados de severidade ERROR e envia todos os achados para o GitHub Code Scanning.

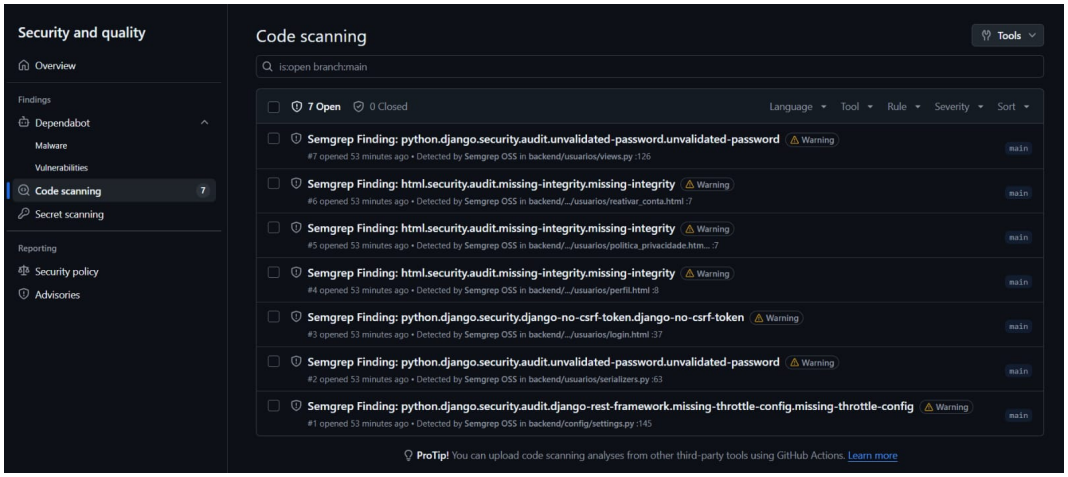
DevSecOps Avançado (`.github/workflows/devsecops-avancado.yml`)

- **DAST (OWASP ZAP):** full scan contra a aplicação em execução (PostgreSQL + Django + Nginx/HTTPS).
- **Privacidade (Bearer):** detecta vazamento de dados pessoais no código (LGPD).
- **Supply chain (Trivy):** gera o SBOM CycloneDX da imagem, publicado como artefato.

Proteção da branch `main`

Ruleset ativo que exige Pull Request e a aprovação dos jobs `Secrets Scanning - TruffleHog`, `SCA - Trivy` e `SAST - Semgrep` antes do merge. Também bloqueia force push e exclusão da branch, sem exceções na lista de bypass.

Evidência 10 — Painel de Code Scanning com alertas/achados listados.



Security and quality

Overview

Findings

Dependabot

Malware

Vulnerabilities

Code scanning 7

Secret scanning

Reporting

Security policy

Advisories

Code scanning

Tools

is:open branch:main

7 Open 0 Closed Language Tool Rule Severity Sort

Semgrep Finding: python.django.security.audit.unvalidated-password.unvalidated-password Warning
#7 opened 53 minutes ago • Detected by Semgrep OSS in backend/usuarios/views.py :126 main

Semgrep Finding: html.security.audit.missing-integrity.missing-integrity Warning
#6 opened 53 minutes ago • Detected by Semgrep OSS in backend/.../usuarios/reativar_conta.html :7 main

Semgrep Finding: html.security.audit.missing-integrity.missing-integrity Warning
#5 opened 53 minutes ago • Detected by Semgrep OSS in backend/.../usuarios/politica_privacidade.htm... :7 main

Semgrep Finding: html.security.audit.missing-integrity.missing-integrity Warning
#4 opened 53 minutes ago • Detected by Semgrep OSS in backend/.../usuarios/perfil.html :8 main

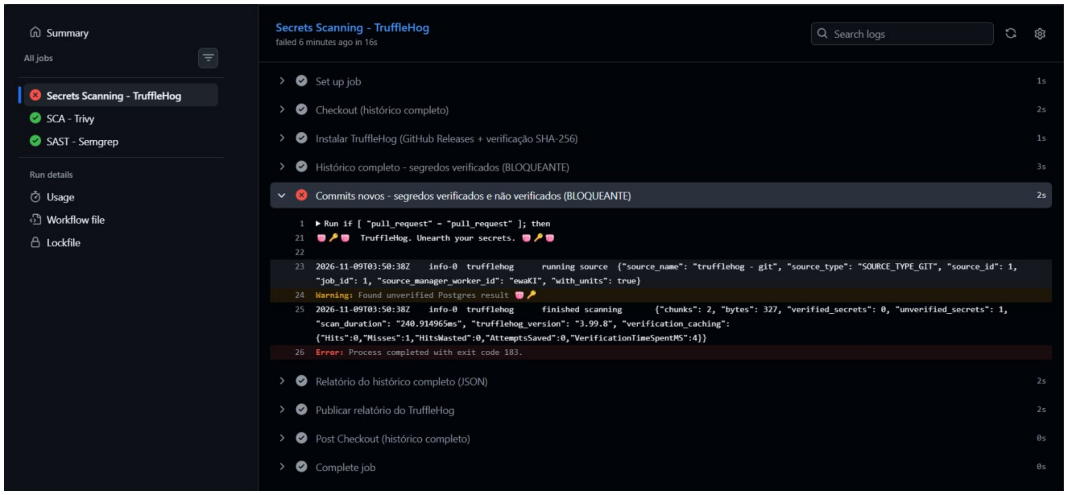
Semgrep Finding: python.django.security.django-no-csrf-token.django-no-csrf-token Warning
#3 opened 53 minutes ago • Detected by Semgrep OSS in backend/.../usuarios/login.html :37 main

Semgrep Finding: python.django.security.audit.unvalidated-password.unvalidated-password Warning
#2 opened 53 minutes ago • Detected by Semgrep OSS in backend/usuarios/serializers.py :63 main

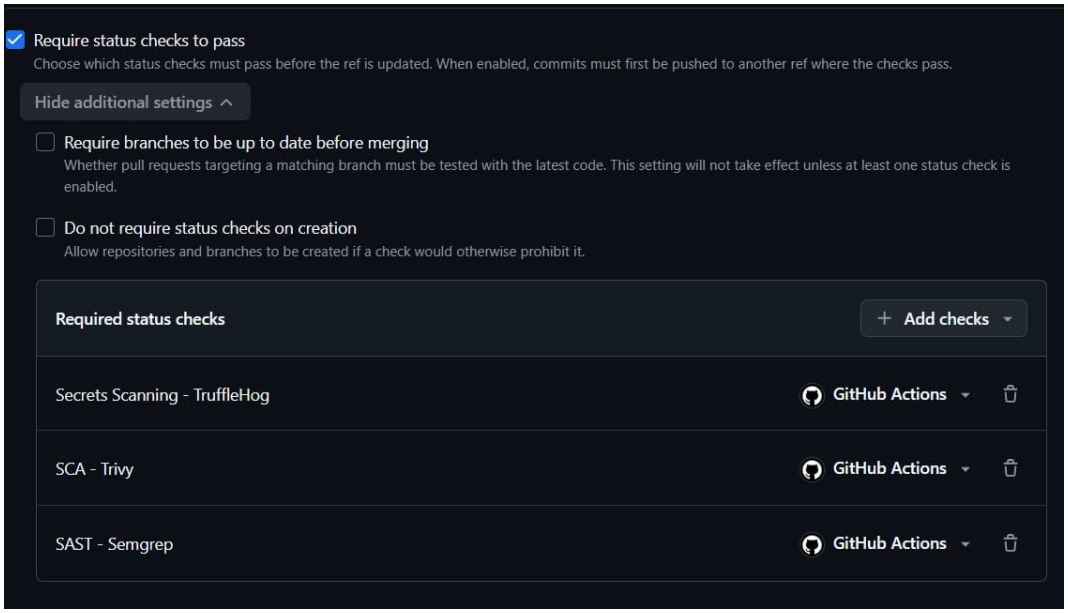
Semgrep Finding: python.django.security.audit.django-rest-framework.missing-throttle-config.missing-throttle-config Warning
#1 opened 53 minutes ago • Detected by Semgrep OSS in backend/config/settings.py :145 main

ProTip! You can upload code scanning analyses from other third-party tools using GitHub Actions. Learn more

Evidência 11 — Detalhe da execução e saída dos jobs de análise de segurança.



Evidência 12 — Ruleset: seleção de status checks obrigatórios TruffleHog, Trivy e Semgrep.



Evidência 13 — Pull Request de teste com credencial falsa: TruffleHog falha e aparece como check obrigatório; os demais checks mostrados passam.

